ASSIGNMENT-19.3

K . Jashwanth

BATCH-42

2303A52300

TASK-1: Python to C++ Conversion

PROMPT:

Convert the following Python class into an equivalent C++ class. The class represents a Student with attributes like name and marks, and includes methods to display student details. Ensure proper use of constructors, appropriate data types, and access specifiers (public/private). The C++ code should be complete, compilable, and produce the same output as the Python version.

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)


# Demonstrate Python class
student_py = Student("Alice", 20, 95)
student_py.display()
```

CODE:

```python
# Define the Student class
class Student:
    # Constructor: Initializes the Student object
    # Takes name, age, and marks as parameters
    def __init__(self, name_param, age_param, marks_param):
        self.name = name_param     # Assign name
        self.age = age_param       # Assign age
        self.marks = marks_param   # Assign marks
    # Public method to display student information
    def display(self):
        print("Name:", self.name)    # Print name
        print("Age:", self.age)      # Print age
        print("Marks:", self.marks)  # Print marks
# Main program
# Create object of Student class
s = Student("Alice", 20, 95)

# Call display function
s.display()
```

OUTPUT:

```
Name: Alice
Age: 20
Marks: 95
```

JUSTIFICATION:

This task focuses on using AI tools to translate code from Python to C++ while understanding the differences between programming languages. A Python Student class is converted into an equivalent C++ class by properly handling constructors, data types, and access specifiers. The translated code is then compiled and executed to ensure it produces the same output. The task also helps in learning syntax and structural differences between languages and highlights the importance of debugging and human verification for accuracy.

TASK-2: Java to Python Function Conversion

PROMPT:

Convert the following Java method that checks whether a number is prime into an equivalent Python function. Ensure the Python code follows proper Python syntax, uses indentation instead of braces, and maintains the same

logic. The function should be tested with multiple inputs to verify correctness.

```java
public class PrimeCheck {

    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] testNumbers = {0, 1, 2, 3, 4, 5, 10, 17, 19, 20};

        for (int num : testNumbers) {
            if (isPrime(num)) {
                System.out.println(num + " -> Prime");
            } else {
                System.out.println(num + " -> Not Prime");
            }
        }
    }
}
```

CODE:

```python
def isPrime(n):
    if n <= 1:
        return False
    for i in range(2, n // 2 + 1):
        if n % i == 0:
            return False
    return True

# Test cases
print(isPrime(2))
print(isPrime(10))
print(isPrime(17))
```

OUTPUT:

```
True
False
True
```

JUSTIFICATION:

In this task, a Java method used to check whether a number is prime is converted into Python. The logic remains the same by checking divisibility from 2 to half of the number. In Python, syntax differences such as removing data types, using indentation instead of braces, and replacing true/false with True/False are applied. The Python function is tested with multiple inputs to ensure it produces correct results, confirming that the translation is accurate and functionally equivalent.

TASK-3: Pseudocode to Python Implementation

PROMPT:

Convert the following pseudocode for sorting numbers using Bubble Sort into an equivalent Python program. Ensure the Python code follows proper syntax, uses loops correctly, and produces the sorted output. Also test the program with sample input lists to verify correctness.

PSEUDO CODE:

```
BEGIN
  INPUT list of numbers
  FOR i from 0 to n-1
    FOR j from 0 to n-i-2
      IF list[j] > list[j+1]
        SWAP list[j] and list[j+1]
      ENDIF
    ENDFOR
  ENDFOR
  PRINT sorted list
END
```

CODE:

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
# Sample input
numbers = [64, 34, 25, 12, 22, 11, 90]
# Sorting
sorted_numbers = bubble_sort(numbers)
# Output
print("Sorted list:", sorted_numbers)
```

OUTPUT:

```
•••  Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

JUSTIFICATION:

In this task, a Bubble Sort pseudocode is converted into a Python program. The logic remains the same, where nested loops are used to repeatedly compare adjacent elements and swap them if they are in the wrong order. Python syntax replaces pseudocode constructs by using for loops, indentation, and tuple swapping instead of a temporary variable. The program is tested with a sample list to ensure correctness, and the output displays the sorted list, confirming successful implementation.

TASK-4: SQL to Pandas Query

PROMPT:

Convert the following SQL query into an equivalent Pandas DataFrame operation in Python. Ensure the Python code correctly filters and selects the required columns. The result should match the SQL query output when tested on a sample dataset.

SELECT name, salary

FROM employees WHERE

salary > 50000;

CODE:

```
import pandas as pd
# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}
df = pd.DataFrame(data)
# Equivalent Pandas query
result = df[df['salary'] > 50000][['name', 'salary']]
# Output
print(result)
```

OUTPUT:

```
      name  salary
0    Alice   60000
2  Charlie   70000
```

JUSTIFICATION:

In this task, a SQL query is translated into an equivalent Pandas operation in Python. The SQL SELECT statement is mapped to selecting specific columns in the DataFrame, and the WHERE clause is implemented using conditional filtering (df['salary'] > 50000). The filtered result is then displayed using Pandas syntax. A sample DataFrame is created to test the code, ensuring that the output matches the SQL query result, demonstrating correct translation and functionality.

TASK-5: Real-Time Application: Algorithm Translation Across

PROMPT:

Translate a Python searching algorithm (Linear Search) into an equivalent C program. Ensure that the logic remains the same while adapting syntax differences such as loops, data types, and input/output functions. The translated code should be tested to verify that both versions produce the same output.

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Sample input
arr = [10, 20, 30, 40, 50]
key = 30

# Function call
result = linear_search(arr, key)

# Output
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

CODE:

```c
#include<stdio.h>
int linear_search(int arr[],int n,int key){
    for(int i=0;i<n;i++){
        if(arr[i]==key){
            return i;
        }
    }
    return -1;
}
int main(){
    int arr[]={10,20,30,40,50};
    int n=5;
    int key=30;
    int result=linear_search(arr,n,key);
    if(result!=-1)
        printf("Element found at index:%d\n",result);
    else
        printf("Element not found\n");
    return 0;
}
```

OUTPUT:

```
Element found at index: 2
```

JUSTIFICATION:

This prompt ensures that the algorithm is translated correctly between two different programming languages while maintaining the same logic. It highlights key differences such as Python's simple syntax versus C's need for explicit data types and memory handling. Testing both versions helps verify correctness and consistency of output. This task also demonstrates real-world challenges like syntax differences, lack of built-in functions in C, and manual handling of arrays.